ODS v2 and the episode event stream: team pack

Oct 19, 2026 · @vijay

ods_v2 (53 tables, no personal data) is built and reconciled in Dev_CGP; the new episode event stream has Motor acquisition and renewal ported and tested locally.

In one page

- **ods_v2** is the rebuilt ods layer: 53 tables that reassemble every dlk / edw table the ads / prd assets read, under one schema, with history. Nothing in it is derived and nothing in it is personal data. It is **config-driven**: one YAML entry per table generates the Fabric notebooks. It is built in Dev_CGP and **reconciled** against the current ods and the sources; policy attributes are identical on all 4,655,071 matched policies.
- **ads_v2.EventStream** is the episode event stream rebuilt **from ods_v2 only**. It replaces notebooks 04 (Populate Episode Event Stream), 05 (Attach SCV ID) and 06 (Episode Numbering), with the same config-driven build. Motor (91 event types) and Home (74) are ported with the agreed fixes and tested locally, and their notebooks are uploaded to Dev_CGP (folder 03 - EADM / Episodes), not yet run.
- **For the DataScience Exploration workspace**, ods_v2 / ads_v2 will be exposed read-only as schema `cgp` through shortcuts. The `pii` schema is never exposed.
- **What we need from you**: the last section (decisions and checks).

Architecture

Client data is reassembled once in ods_v2; everything derived is built in ads_v2 from ods_v2 alone.

Layer	Holds	Built from
dlk	replica of the SQL Server ODS schema	client source
edw	copy of the SQL Server DWH schema	client source
pii	SCV personal data	read inside Fabric only, to make salted hash keys; never copied out
ods_v2	client data reassembled: nothing derived, no personal data, history	dlk, edw (+ pii as hash keys)

Layer	Holds	Built from
ads_v2	event stream, SCV links, episodes	ods_v2 only (+ the SCV link `ods.scveadmlinkfinal`)
prd	analytics assets	ads_v2

- **Promotion**: Dev_CGP → UAT → Prod. The Fabric `dwh` schema (a failed earlier build) is ignored.
- **Sources** come from a switch in `settings.yaml`. The **frozen** set (dlk, edw, `dlk_bak_20260914` for CDA, ods for the carried tables, pii) is used for the one-off dev build. The **prod** set (prod_dlk, prod_edw, prod_pii) takes over when the live feed lands.
- **edw** is a source for now. The plan is to derive the edw-sourced tables from dlk with the DWH Build logic, so edw is used only for reconciliation.
- **Policy objects** copy the edw structure, limited to the fields in use, with surrogate keys (`PolicyId`, `QuoteId`, …).

How ods_v2 is built

One YAML entry per table is the only thing anyone edits; a generator writes the Fabric notebooks from it.

Piece	Where	What it does
Settings	`ods_build/v2/settings.yaml`	Target schema, source sets, cut-off (2021-04-01), the PII column-name pattern, the notebook list and run order
Table config	`ods_build/v2/entities/*.yaml`	One entry per table: notebook, grain, business key, surrogate key, history, cut-off, source, description, SQL steps. **The only thing to edit.**
Generator	`ods_build/v2/build_notebook.py`	Writes the notebooks, `ODS_v2_all.sql` (all SQL in one file, for review) and `ODS_v2_Entity_Model.md` (the catalogue)

Piece	Where	What it does
Notebooks	`ODS_v2_00_Setup`, `10_Policy`, `20_Quote`, `21_Quote_Motor`, `22_Quote_Home`, `23_Quote_Travel`, `24_Quote_Van`, `30_Interaction`, `40_Reference`	Setup holds the settings, the hash salt and the hash / redact functions; every build notebook runs it first with `%run`
Checks	last cell of every notebook	Unique, non-null business key; no PII-named columns; no email or mobile values in text columns. Results go to `ods_v2.OdsBuildLog`; any failure fails the notebook
Run log	`ods_v2.OdsRunLog`	One row per batch, notebook and status (started / succeeded / failed)

House style. One SQL statement per cell and no CTEs: intermediate steps are `stg.odsv2_*` tables. Every cell has a header saying what it builds, what it reads, its grain and key. Every cell is `create or replace`, so re-running from a failed cell is always safe.

Run order. 10_Policy, then 20_Quote, then 21–24, 30 and 40 (in parallel). Starting notebooks from another notebook (runMultiple / notebook.run) fails in Fabric, but each notebook runs fine as a job or by hand. Next step: a Fabric pipeline chains them; until then, run them by hand in that order.

If a cell fails (this note also sits at the top of every notebook):

1. Read the failing cell's header.
2. Fix the cause. Usually a source changed (a renamed column) or a key broke (duplicates); the checks cell names the table and the likely cause.
3. Re-run from the failed cell (after a session restart, run the first three cells first).
4. Copy the fix into the YAML and regenerate. A fix made only in Fabric is lost at the next generation; the drift check (`checks/drift_check.py`) lists such cells.

Keys. Surrogate keys are `row_number()` over the business key for this one-off build. The incremental build will use a persistent source-key → surrogate-key map, as the prod SCV build does.

Personal data

ods_v2 holds no personal data: contacts are salted hash keys, free text is redacted, and the rest is never taken.

- **Salted SHA-256 keys only** (`*Key` columns): email (lower / trim), phone (Irish national form, so +353 / 00353 / 08 all match), driving licence and vehicle registration (upper case, no spaces; licence `123456` = null). Keys match across tables and across ods and ads.
- **Not taken**: names, addresses, dates of birth (an age at cover start instead), message bodies, free-text notes, full Eircodes. The Eircode routing key (first 3 characters) is kept as `*EircodeArea`.
- **Redacted**: free-text and id columns that can hold typed-in contact details go through `REDACT()`, which replaces an email with `[email]` and a mobile with `[phone]`: InsPolNumber, ArReference, Reasons, PageDisplayErrorMessage, TransferToId, InteractionValue, PagePath and Home QS PolicyReference.
- **Plain codes**: policy, client and quote codes are kept as they are (internal to Fabric).
- **The salt** sits in the `ODS_v2_00_Setup` parameters for now and moves to Key Vault later. It must never change once ods_v2 is in use, because every key would change with it.
- **pii schema**: read only inside Fabric to build SourceCustomer (hash keys only). Builds and tests that touch pii return summaries only; no raw values leave Fabric.

The 53 ods_v2 tables

Event and transaction tables start at the cut-off (2021-04-01); policy tables keep their full history. Every table also carries `OdsBatchId` and `OdsLoadDateTime`. The full catalogue, with a description of each table, is `ods_build/v2/ODS_v2_Entity_Model.md`.

Policy (23)

Table	Grain	Business key	Source
Policy	policy code, latest version	PolicyCode	edw.tbl_ref_policy
PolicyVersion	policy code and record version	PolicyCode + RecordVersion	edw.tbl_ref_policy
PolicyTravel	Mapfre travel policy, latest update	MapfrePolicyId	dlk.ext_travel_policy(_trans)
PolicyTravelTransaction	Mapfre travel policy transaction	TravelTransactionKey	dlk.ext_travel_policy_trans
PolicyMotorProposer	policy code and record version	PolicyCode + RecordVersion	carried from ods (no edw source in Fabric)

Table	Grain	Business key	Source
PolicyMotorVehicle	policy code and record version	PolicyCode + RecordVersion	carried from ods
PolicyHome	policy code and record version	PolicyCode + RecordVersion	carried from ods
PolicySale	policy code and reporting sale date	PolicyCode + ReportingSaleDate	edw.tbl_fact_policy_sales
PolicyRenewal	policy code and last-year renew date	PolicyCode + LYPolicyRenewDateAdj	edw.tbl_fact_policy_renewals_rebuilt
PolicyMonthEnd	policy code and month end	PolicyCode + EffectiveDate	edw.tbl_fact_policy_mvt
PolicyCancellation	cancellation record	PolicyCancellationId	edw.tbl_fact_policy_mtc
PolicyTransaction	Relay accounts transaction	PolicyCode + PostingDate + TransSeq + …	dlk.rbsdata2_newarchive + premium type map
PolicyMta	policy, history sequence and type	PolicyCode + HistorySeq + HistoryType	dlk.rbsdata2_mta_history
PolicyClaimMotor	policy, claim date, type, value	PolicyCode + ClaimDateRaw + ClaimType + …	dlk.rbsdata2_motor_clm
PolicyClaimHome	home claim	HomeClaimId	dlk.ext_home_qs_claimdetails (+ policydetails)
ClientPortfolio	Relay portfolio and policy	PortfolioId + PolicyCode	dlk.relay_policy + relay_portfolio
PolicyDocumentEvent	Relay document event and policy	EventId + DocumentId + PolicyCode	dlk.relay_eventmodel_* / documentation_*
RenewalPortalEvent	portal event (login / payment)	EventType + EventId	dlk.appliedrenewals_*
DocumentWorkflowStatus	MyChill workflow record	Pk + Rk	dlk.mychillworkflow_documentstatus
DocumentUpload	MyChill upload record	Pk + Rk	dlk.mychill_newuploaddocumentevents
DocumentChaseHome	file date and policy	FileDate + PolicyCode	edw MyChill chase snapshot (home)
DocumentChaseMotorVan	file date and policy	FileDate + PolicyCode	edw MyChill chase snapshot (motor / van)
PolicyHomeQsEvent	Home QS transaction (risk id)	HomeRiskId	dlk.ext_home_qs_policydetails

Quote (21)

Table	Grain	Business key	Source
Quote	source system and quote reference (all quote systems)	SourceSystemId + SourceSystemQuoteReference	MFQ, QuotationStorage, HFQ, Home QS, travel, van
QuoteMotor	MFQ quote guid	GuidKey	dlk.mfq_quotequery (+ drivers, client steps)

Table	Grain	Business key	Source
QuoteMotorSummary	MFQ quote guid (latest month end)	GuidKey	edw.tbl_fact_mfq
QuoteMotorPrice	MFQ guid and insurer (cheapest row)	GuidKey + Insurer	dlk.mfq_quoteresponses + quotesummary
QuoteMotorPayment	MFQ payment attempt	PaymentId	dlk.mfq_payments
QuoteMotorStep	MFQ client step event	ClientStepId	dlk.mfq_clientsteps
QuoteMotorBotSession	MFQ monitoring session	MonSessionId	dlk.mfq_mon_session
QuoteMotorDriver	quote and driver PRN	SourceSystemId + SourceSystemQuoteReference + DriverPRN	MFQ, QuotationStorage
QuoteMotorVehicle	motor quote	SourceSystemId + SourceSystemQuoteReference	MFQ, QuotationStorage
QuoteHome	home quote	SourceSystemId + SourceSystemQuoteReference	HFQ, Home QS
QuoteHomePrice	HFQ response row	ResponseQuoteId	dlk.hfq_response_quotes
QuoteHomePaymentRequest	HFQ payment request	GlobalPaymentReference	dlk.hfq_paymentrequest_request
QuoteHomePaymentResponse	HFQ payment response	PaymentResponseId	dlk.hfq_paymentresponse_response
QuoteHomeSale	HFQ sale record (latest load)	SaleRecordId	dlk.hfq_sales
QuoteTravel	travel quote (incomplete quotes: hashed email + travel date)	SourceSystemQuoteReference	dlk.ext_travel_quotes
QuoteVan	van quote record, latest partition	RowKey	dlk.vanquotedetails
QuoteVanJourney	van quote page event	QuoteReference + SessionId + EventTimestamp + ClickCode	dlk.vanquotejourney
QuoteVanPayment	van payment event	QuoteReference + SessionId + PaymentDateTime + ClickCode	dlk.vanpaymentjourney
PolicyQuoteLink	warehouse quote event	QuoteEventID	edw.tbl_fact_policy_quotes
WebQuoteClick	MFQ guid and GA click	GuidKey + ClientID + ClickOrder + EffectiveDate	edw.tbl_fact_ga_clicks_ga4
WebRetrieveHit	retrieve-page hit with a reference	ClientID + ClickOrder + HitDateTimeUtc	edw.tbl_fact_ga_clicks_ga4

Interaction (8) and reference (1)

Table	Grain	Business key	Source
CallSession	Genesys conversation and session	ConversationId + SessionId	dlk.genesys_session_summary
TelephonyCall	call id and sequence	CallIDKey + SeqNo	edw.tbl_fact_telephony_call_detail
MessageEvent	Xtremepush message event	SourceFeed + CampaignId + ProfileId + InteractionType + EventDateTime + …	dlk.ext_xtremepushresults(_policy)
Campaign	campaign id and name	CampaignId + CampaignName	from MessageEvent
ConsentAction	Mercury consent action	ActionTrackId	dlk.merc_*
CdaMessage	CDA message (type by pattern; body not held)	MessageId	dlk_bak_20260914.sql07cda_messages
WebPageView	GA4 page view	ClientID + ClickOrder + EffectiveDate + EventDateTime	dlk.ga4_page_views_daily
SourceCustomer	SCV source customer (hash keys only)	SourceSystemId + SubSourceSystemId + SourceSystemCustomerId	pii.customer_data_assembled (+ addresses)
Calendar	day	DayDate	edw.tbl_ref_calendarlookup

Source system ids across ods_v2 and ads_v2: 1 Relay, 2 MFQ, 3 HFQ, 4 Home QS, 5 Mapfre, 7 Van, 8 Travel quotes, 9 QuotationStorage, 20 Genesys, 21 Xtremepush, 22 Mercury, 23 CDA.

Testing and reconciliation

Every split notebook has run in Dev_CGP with all checks passing, and ods_v2 reconciles to the current ods and its sources.

- **Config tests** (no Spark, run anywhere; 141): every entity is complete, reads only columns that exist in the source catalogue, has no PII-named outputs, and reads only from its own notebook or those it depends on. They also check that every non-SQL cell

starts with `%%pyspark`.

- **Hashing and build tests** (local Spark 3.5 + Delta in WSL; 52 + 291): the hash functions match Spark `sha2`, every generated notebook runs end to end, and every check passes. They run against a PII-free sample of the sources (contact details replaced by fakes; the extractor refuses to write if anything slips through) plus synthetic pii.
- **Fabric builds (Dev_CGP)**:
  - Run 1 found what the tests could not: legitimately empty key parts, two source duplicate sets, and emails or mobiles inside free-text or id values. All three were fixed.
  - Run 2 passed every check on all tables in 82 minutes.
  - The split-notebook run on 18 October: 22_Quote_Home failed its PII check (about 40 typed-in emails and some mobiles in Home QS `PolicyReference`). It was fixed with `REDACT()` and re-run, and passed. Every other notebook completed with all checks passing.
- **Reconciliation** (`out/ods_design/04_reconciliation.md`, accepted):
  - Policy, PolicyTravel, ClientPortfolio, QuoteTravel and QuoteVan keys match the current ods exactly.
  - Policy attributes (status, dates, premium, client, version) differ on 0 of 4,655,071 matched policies.
  - PolicyDocumentEvent matches all 107,662,978 events from the cut-off.
  - ods_v2 has more quotes than the current ods: it adds HFQ, Home QS and van quotes, and MFQ quotes the current ods misses. It starts at the cut-off, which accounts for every quote found only in the current ods.
  - Intended differences: MFQ times are in UTC; MFQ drivers come from the latest query (an MFQ source bug); the premium-type mapping fix raised mapped transactions from 151k to 2.45M.

The episode event stream

Notebooks 04, 05 and 06 become one config-driven build that reads ods_v2 only; Motor acquisition and renewal are done.

From three notebooks to one build

Today	ads_v2
Notebook 04 inserts ~300 event types into `ods.EpisodeEventStream` from dlk, edw, ods, stg and lkp, with hard-coded dates	**One YAML entry per event type** (`ads_build/v2/families/*.yaml`) reading **ods_v2 only**. The generator writes one `stg.adsv2_evt_<code>` table per type, and a union step builds `ads_v2.EventStream`
Build window in `stg.episodeeventstream_buildconfig` plus literals in many cells	`start_time`, `end_time`, `effective_date` in `ads_build/v2/settings.yaml` (notebook parameters). A test rejects any date written into the SQL
Event descriptions typed in each cell (with typos)	From the version-controlled seed `seeds/ChillEpisodeTypes.csv` (export of `lkp.chillepisodetypes`, 736 step ids), by code
EventId = max + row_number, different on every run	`xxhash64(code, grain, references, minute)`: the same id for the same event on every run; the checks fail on any collision
Raw emails, phones and user ids in the stream	Hash keys only. The ods checks (PII names, email / mobile values) run on ads_v2 too
Notebook 05 links SCV by several routes, including `scv_customer_key`	`ods.scveadmlinkfinal` is the **only** policy / quote / client → SCV link (travel via `'M'` + MapfreCustomerId). Contacts link through a point-in-time contact rule
Unlinked events silently get a negative key	Every unlinked reference is collected in **`ads_v2.ScvLinkExceptions`**, for someone to check with the client
Notebook 06 drops events whose code is missing from the lookup	Missing codes are reported, not dropped

SCV contact rule (agreed 2026-10-18)

- **Quote contacts** (quote systems 2, 3, 4, 7, 8, 9): the use date is `scveadmlinkfinal.QuoteStartDate`. Against the ods_v2 quote dates: 100% same day for systems 3 / 4 / 8 / 9, 97.2% for MFQ (the rest one day apart: UTC) and 93.8% for van.

- **Client contacts** (Relay clients, travel customers): the warehouse's rule from `DWH.usp_Load_fact_Telephony_Call_Detail`. Match phone 1 / 2 / 3 as at the call date; the primary client is the one with the latest client or policy record change before the call; ties go to MAX(ClientCode).
- At the event time, take the SCV key with the latest use on or before the event; if none, the earliest use within 30 days after. **When live, that look-ahead is provisional**: re-checked for 30 days, and shown in ScvLinkExceptions until it links.

ScvLinkExceptions

One row per reference that did not link: ReferenceType (PolicyCode / ClientCode / QuoteReference / TravelCustomer / EmailKey / PhoneKey / UserIdKey), Reference, SourceSystemId, PolicyTypeGroup, ExceptionReason, EventCount, First / LastEventDate, EventTypes, Priority (1 = policy or sold quote, 2 = a quote with a price or payment, 3 = other), First / LastSeenBatch. Contact references are hash keys; someone with pii access resolves them inside Fabric. Known gaps seed it from day one: 6,237 MFQ quotes and 3,957 travel policies missing from scveadmlinkfinal.

Developer handover: what ads_v2 implements and what is outstanding

Motor and Home are built with every agreed fix that does not need the SCV link, tested locally (0 failed cells, 54,243 events on the sample) and uploaded to Dev_CGP, not yet run in Fabric. The status of each of the 84 issues found in notebook 04 is on the `Event stream issues` tab: 48 fixed, 4 agreed but not built, 27 needing a decision, 5 not yet ported.

Notebooks in Dev_CGP (folder 03 - EADM / Episodes; generated from `ads_build/v2/families/*.yaml` by `build_ads.py`, so fixes go into the YAML):

Notebook	Builds	Run
ADS_v2_00_Setup	Settings, build window, hash salt (the same as ods_v2)	Run by every notebook below
ADS_v2_10_Common	Shared tables: active sales, Relay documents, renewal cycles (finance cycles plus the +/-30-day match), each cycle's re-quotes, HFQ prices and payments	1st
ADS_v2_11_Motor_Acquisition	35 event types	2nd, in parallel with 12-14
ADS_v2_12_Motor_Renewal	56 event types	2nd

Notebook	Builds	Run
ADS_v2_13_Home_Acquisition	28 event types	2nd
ADS_v2_14_Home_Renewal	46 event types	2nd
ADS_v2_19_EventStream	Union into ads_v2.EventStream (one row per EventId), the checks, and ads_v2.AdsFamilyCheck	Last

Rules agreed with Vijay (decision log `out/e1_findings.md`):

Decision	Rule
D-E1.1, R-4	A3 / A3.F1: the SL_10 view's Tot-All-First rule on QuoteMotorSummary; bots by ScraperFlag
R-3 (D-E1.2)	MFQ payments: 1 = abandoned, 4 = failed, 0 / 5 = success
D-E1.15 to D-E1.18	Home acquisition: HEQReference and channel split HA4b / HA4.F1; chases at their send dates; Service / Marketing campaigns; HA0 excludes callers with an HFQ quote in the previous 30 days
D-E1.19	Renewal cycles: the finance PolicyOfferNum / PolicyRetNum, plus the operations +/-30-day window on top
D-E1.20	One event per occurrence at its first time; EventTimeBasis (actual / derived) on every event
D-E1.21, D-E1.26	Lapsed = no renewal by the due date; renewed inside the window = by due + 7 online ('e) Online' or a portal payment) or due + 30 through the call centre (inbound, outbound, back office, Web Assist)
D-E1.24, D-E1.25	Renewal calls: customer through the contact link, policy = the renewal open at the call date (due - 60 to + 30). Call wait friction = a wait over 600 seconds, or abandoned; no wrap-up codes
D-E1.27	Re-quotes: the same customer's quote for the same product, due - 60 to due + 30, compared with the renewal offer; O1 / O2 by how the cycle renewed

Decision	Rule
D-E1.22, D-E1.23	For later journeys: Van quotes = vanquotedetails + vanquotejourney; Travel purchases from the travel policy table, channel from its Source (Web / Call Centre)
18 October	ods.scveadmlinkfinal is the only policy / quote / client to SCV link; outbound sales callbacks = OUTBOUND_ or INBOUND_SALES_* with direction outbound (QD_* out of scope)

Implemented in ads_v2:

- Re-runnable notebooks reading ods_v2 only; one build window from the settings; stable EventId; hash keys, no personal data; real source system ids; descriptions from the seed.
- Notebook 04 bugs fixed: the '2ndCar' literal, the guid in SourcePolicyReference, HR1.E1 never written, HR3.B1.1.F1 on the Motor base, R3.B1.2.F1 / HR3.B1.2.F1 never returning rows, the SL_10 one-off copy, hard-coded dates.
- Motor acquisition: A5.F1 failed payments only; A3 bots by ScraperFlag; A0.F1 wait rule; A3.E1 outbound callback queues.
- Home acquisition: every 15 October decision (rows 59-67 of the register); HA5, HA7.B1.2, HA7.B1.3 and HA7.B1.5 dropped as agreed.
- Renewal (Motor and Home): the shared renewal cycles with the +/-30-day match; lapsed, renewed inside the window and lost from the cycle, one event per cycle; re-quotes per cycle with O1 / O2 / O3 from how the cycle renewed; the call wait rule.
- Output tests per family in ads_v2.AdsFamilyCheck (count, first and last time, events outside the window, no rows, identical to another family).

Outstanding:

- Agreed, not built (4): every key on every row and renewal call attribution (both need the SCV link step); month bucketing removed only from the outcome and re-quote families so far; the one-event-per-occurrence grain likewise only partly applied.
- Decision needed (27): mostly Motor renewal (proxy times such as due - 60, identical steps, R3c reading the inbound channel, R3b counting every renewal posting) and Motor acquisition (A1 counting retrieves, chases at the sale date). Home renewal questions H2-H6 and H11-H13 still use notebook 04's logic.
- Not yet ported (5): Motor and Home MTA, cancellation, document requests, arrears and claims; Van; Travel; the SCV link step, episode numbering (notebook 06) and the parity report against ods.episodeeventstream.

Assumptions to confirm: HA1.F1 matches a retrieve send to the recipient's unpriced HFQ quotes from the 30 days before it; only EQuote Retrieve Day 1 and Day 6 count as Service (a Day 12 campaign, about 28k sends, is left out); lost and "policy lapsed" are stamped at due + 30, "online window closes" at due + 7, lapsed at the due date, each written once that date has passed; the cycle list is the finance list (PolicyOfferNum = 1), and the +/-30-day rule decides only whether a cycle renewed.

After the first Fabric run, check:

1. ads_v2.AdsRunLog: every notebook succeeded; ads_v2.AdsBuildLog: the EventStream checks passed.
2. ads_v2.AdsFamilyCheck: each flag (no rows, outside the window, identical to another family) has an explanation.
3. Read 3 to 5 customers' events in time order: is the story believable?
4. Compare counts per event type with ods.episodeeventstream; every difference should match a fixed issue on the register.

Where things live

What	Path (repo `C:\Workspaces\CGP_analysis`)
ods_v2 config, generator, notebooks, tests	`ods_build/v2/` (README there)
ods_v2 catalogue (every table, described)	`ods_build/v2/ODS_v2_Entity_Model.md`
ads_v2 config, generator, notebooks, tests	`ads_build/v2/` (`settings.yaml`, `families/`, `seeds/`, `build_ads.py`, `test/`)
Design notes	`out/ods_design/` (01 demand inventory, 04 reconciliation, 06 mapping, 08 event stream design)
Fabric (Dev_CGP)	notebooks `ODS_v2_00_Setup` … `ODS_v2_40_Reference`; schema `ods_v2`; logs `ods_v2.OdsRunLog` / `OdsBuildLog`

Decisions and checks needed

1. **Orchestration**: a Fabric pipeline to chain the split notebooks (10 → 20 → 21–24 / 30 / 40). Notebook-launched orchestration fails in Fabric.

2. **Batch id**: a job started through the API does not pass `batch_id` into the notebook, so each notebook logs under its own id. To be fixed with the pipeline.
3. **Open items with Vijay**: the SCV link gaps (`07_scv_link_gaps_mfq.csv`, `07_scv_link_gaps_travel.csv`); reinstating `tbl_fact_policy_renewals`; copying `DWH.tbl_ref_PolicyStatus` into `lkp`; cleaning up `ods.rapier*key` (they hold raw PII) and the old `ODS_Build_v2` / `ODS_v2_Run` notebooks.
4. **Later**: the salt in Key Vault; a persistent key map and incremental loads; the prod source switch; GDPR processing upstream of dlk / edw; deriving the edw-sourced tables from dlk.